Exp4.Network Topology Creation using ns3

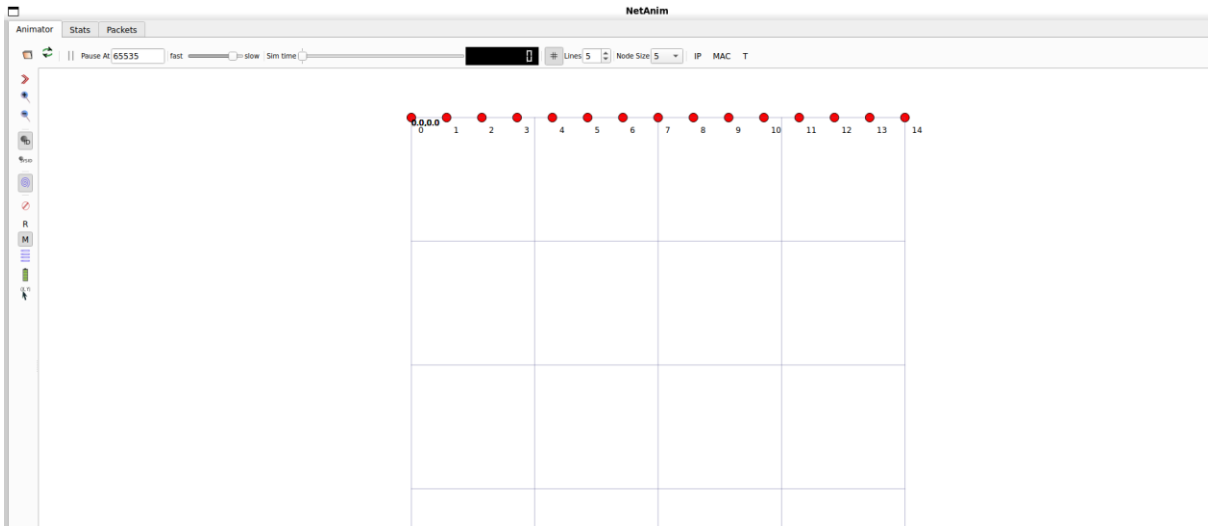
Muthu Venkatesh M

24BCE1389

GITHUB LINK: https://github.com/muthuvenkatesh-24/ns-3

BUS TOPOLOGY

A bus topology is implemented in ns-3 using the CSMA protocol, allowing all nodes to communicate over a common transmission medium. The topology is recorded as an XML animation trace and displayed using the NetAnim visualization tool.



CODE:

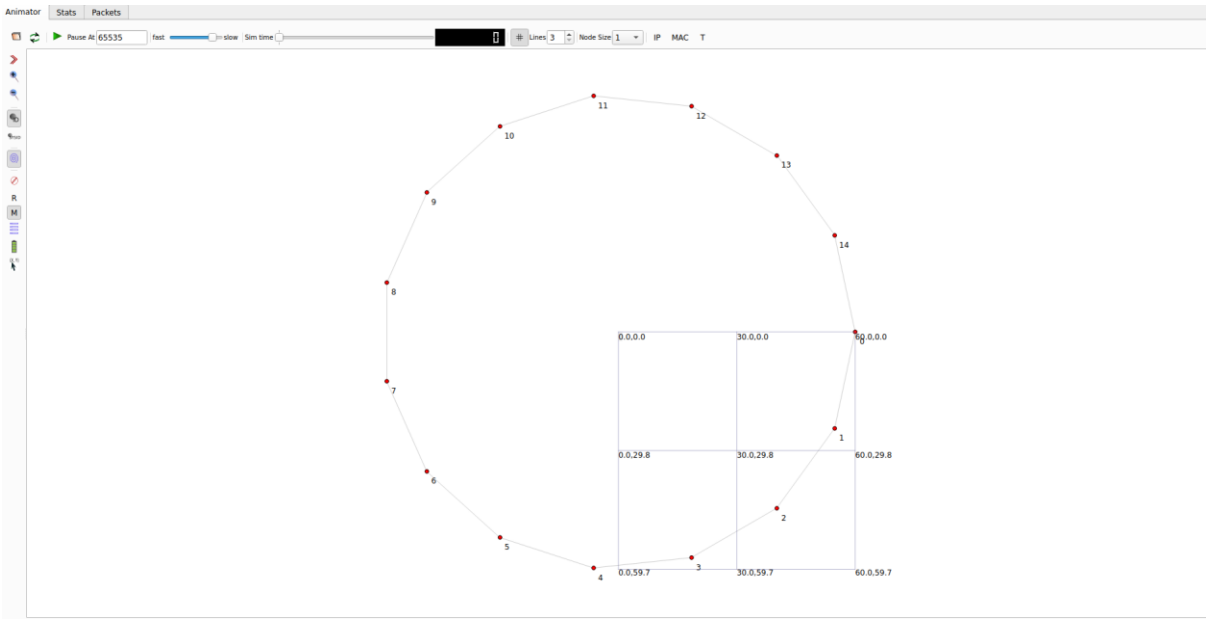
```
#include "ns3/core-module.h"

#include "ns3/network-module.h"

#include "ns3/internet-module.h"

#include "ns3/csma-module.h"
```

```
#include "ns3/netanim-module.h"
#include "ns3/mobility-module.h"

using namespace ns3;

int main () {

  NodeContainer nodes;
  nodes.Create (15);

  CsmaHelper csma;
  csma.SetChannelAttribute ("DataRate", StringValue ("100Mbps"));
  csma.SetChannelAttribute ("Delay", TimeValue (NanoSeconds (6560)));

  csma.Install (nodes);

  InternetStackHelper internet;
  internet.Install (nodes);

  // Mobility for clean layout
  MobilityHelper mobility;
  mobility.SetMobilityModel ("ns3::ConstantPositionMobilityModel");

  Ptr<ListPositionAllocator> pos = CreateObject<ListPositionAllocator>();
  for (uint32_t i = 0; i < 15; i++) {
    pos->Add (Vector (i * 20.0, 0.0, 0.0));
  }
```

```
  mobility.SetPositionAllocator (pos);
  mobility.Install (nodes);

  AnimationInterface anim ("bus.xml");

  Simulator::Stop (Seconds (10));
  Simulator::Run ();
  Simulator::Destroy ();
  return 0;
}
```

Ring topology

In a ring topology, each node is connected to exactly two other nodes, forming a closed loop. Data travels sequentially around the ring.



CODE:

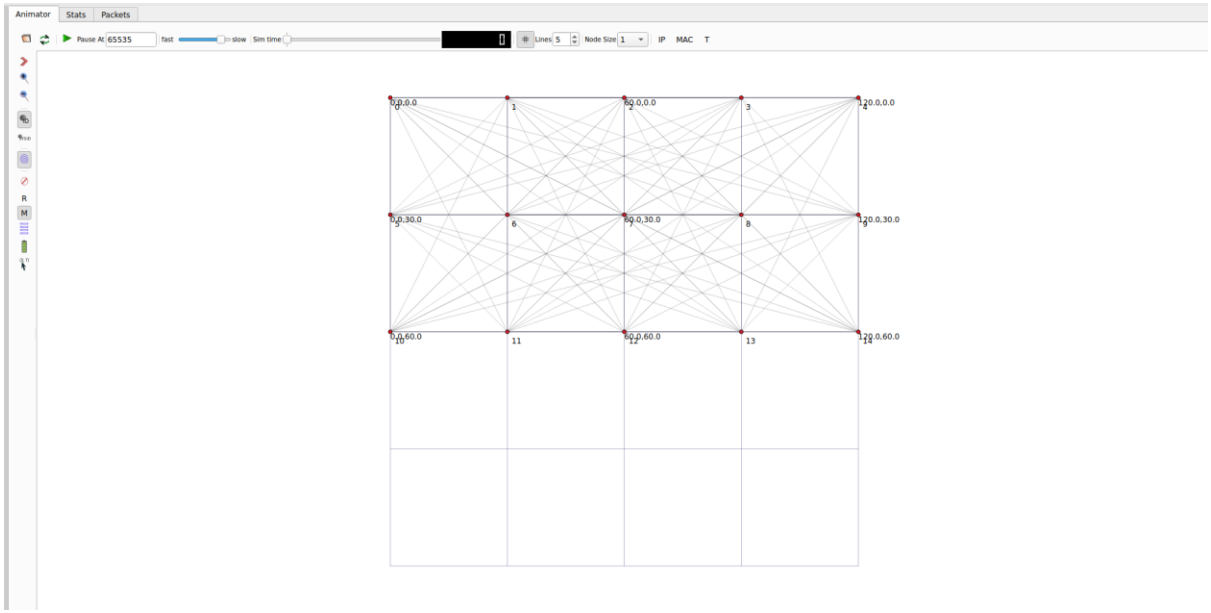
```
#include "ns3/core-module.h"
```

```
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/netanim-module.h"
#include "ns3/mobility-module.h"
#include <cmath>

using namespace ns3;

int main () {

  NodeContainer nodes;
  nodes.Create (15);

  InternetStackHelper internet;
  internet.Install (nodes);

  PointToPointHelper p2p;
  p2p.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));
  p2p.SetChannelAttribute ("Delay", StringValue ("2ms"));

  // Connect nodes in a ring
  for (uint32_t i = 0; i < 14; i++) {
    p2p.Install (NodeContainer (nodes.Get (i), nodes.Get (i + 1)));
  }
  p2p.Install (NodeContainer (nodes.Get (14), nodes.Get (0)));

  // Mobility: circular layout
```

```
  MobilityHelper mobility;
  mobility.SetMobilityModel ("ns3::ConstantPositionMobilityModel");

  Ptr<ListPositionAllocator> pos = CreateObject<ListPositionAllocator>();
  double radius = 60.0;

  for (uint32_t i = 0; i < 15; i++) {
    double angle = 2 * M_PI * i / 15;
    pos->Add (Vector (radius * cos(angle), radius * sin(angle), 0));
  }

  mobility.SetPositionAllocator (pos);
  mobility.Install (nodes);

  AnimationInterface anim ("ring.xml");

  Simulator::Stop (Seconds (10));
  Simulator::Run ();
  Simulator::Destroy ();
  return 0;
}
```

MESH TOPOLOGY

In a mesh topology, each node is connected to every other node, providing maximum redundancy and fault tolerance.



CODE:

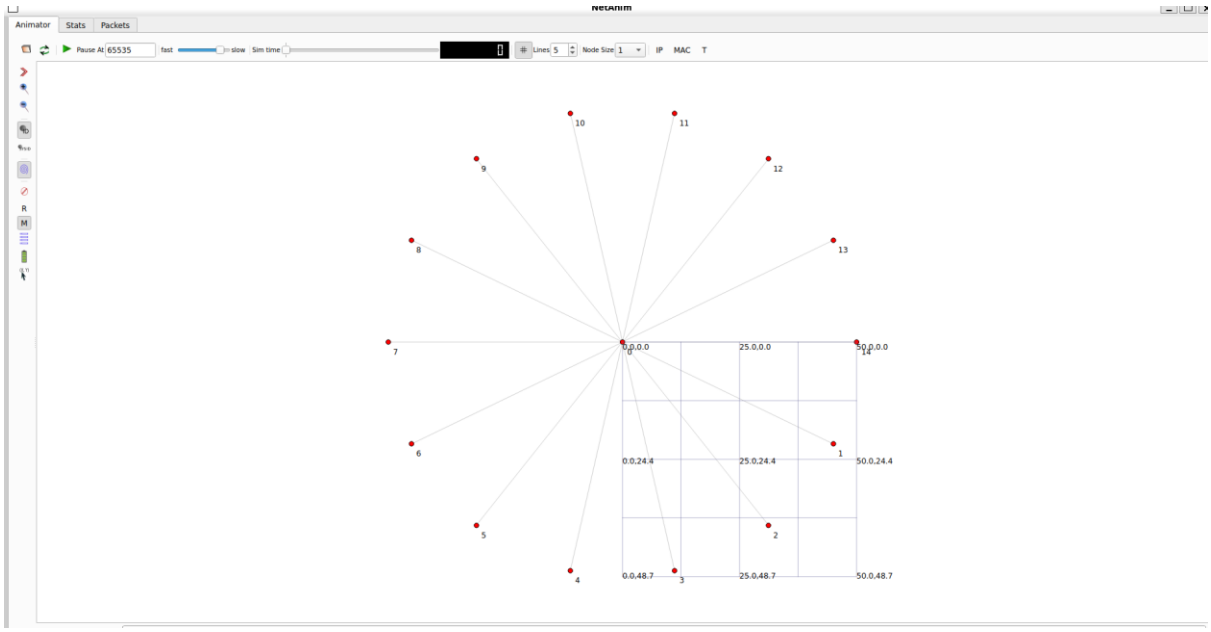
```
#include "ns3/core-module.h"

#include "ns3/network-module.h"

#include "ns3/internet-module.h"

#include "ns3/point-to-point-module.h"

#include "ns3/mobility-module.h"

#include "ns3/netanim-module.h"


using namespace ns3;
```

```
int main ()
{
  NodeContainer nodes;
  nodes.Create (15);

  PointToPointHelper p2p;
  p2p.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));
  p2p.SetChannelAttribute ("Delay", StringValue ("2ms"));

  for (uint32_t i = 0; i < 15; i++)
  {
    for (uint32_t j = i + 1; j < 15; j++)
    {
      NodeContainer pair (nodes.Get (i), nodes.Get (j));
      p2p.Install (pair);
    }
  }

  MobilityHelper mobility;
  mobility.SetMobilityModel ("ns3::ConstantPositionMobilityModel");

  Ptr<ListPositionAllocator> pos = CreateObject<ListPositionAllocator> ();

  pos->Add (Vector (0, 0, 0));
  pos->Add (Vector (20, 0, 0));
  pos->Add (Vector (40, 0, 0));
  pos->Add (Vector (60, 0, 0));
  pos->Add (Vector (80, 0, 0));
```

```
  pos->Add (Vector (0, 20, 0));
  pos->Add (Vector (20, 20, 0));
  pos->Add (Vector (40, 20, 0));
  pos->Add (Vector (60, 20, 0));
  pos->Add (Vector (80, 20, 0));

  pos->Add (Vector (0, 40, 0));
  pos->Add (Vector (20, 40, 0));
  pos->Add (Vector (40, 40, 0));
  pos->Add (Vector (60, 40, 0));
  pos->Add (Vector (80, 40, 0));

  mobility.SetPositionAllocator (pos);
  mobility.Install (nodes);

  AnimationInterface anim ("mesh.xml");

  Simulator::Stop (Seconds (10));
  Simulator::Run ();
  Simulator::Destroy ();
  return 0;
}
```

STAR TOPOLOGY

In a star topology,all each device is connected directly to the hub

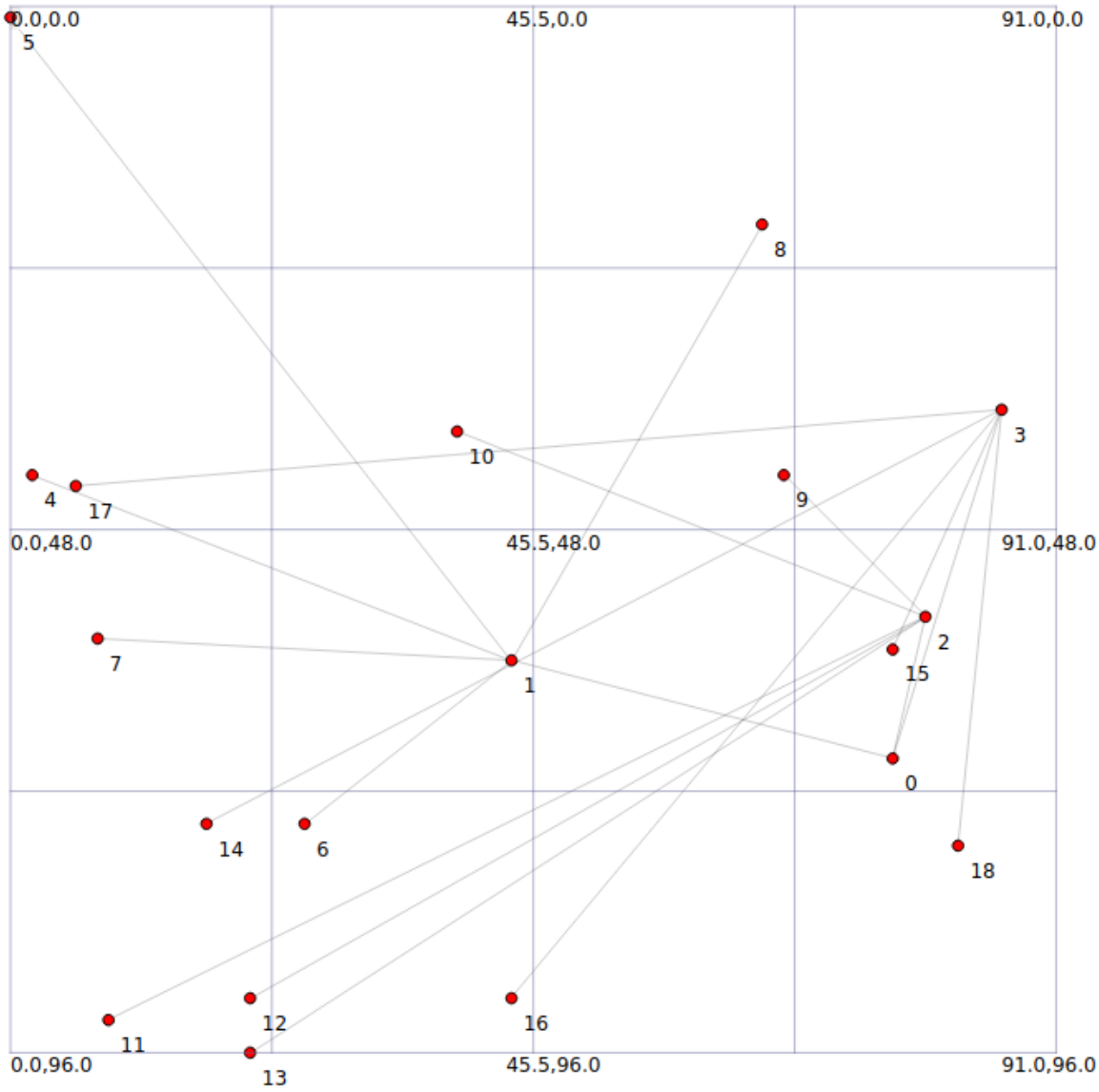



```
#include "ns3/core-module.h"

#include "ns3/network-module.h"

#include "ns3/internet-module.h"

#include "ns3/point-to-point-module.h"

#include "ns3/netanim-module.h"

#include "ns3/mobility-module.h"

#include <cmath>


using namespace ns3;


int main () {
```

```
NodeContainer nodes;
nodes.Create (15);

InternetStackHelper internet;
internet.Install (nodes);

PointToPointHelper p2p;
p2p.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));
p2p.SetChannelAttribute ("Delay", StringValue ("2ms"));

for (uint32_t i = 1; i < 15; i++) {
  p2p.Install (NodeContainer (nodes.Get (0), nodes.Get (i)));
}

MobilityHelper mobility;
mobility.SetMobilityModel ("ns3::ConstantPositionMobilityModel");

Ptr<ListPositionAllocator> pos = CreateObject<ListPositionAllocator>();
pos->Add (Vector (0, 0, 0));

double r = 60;
for (uint32_t i = 1; i < 15; i++) {
  double a = 2 * M_PI * i / 14;
  pos->Add (Vector (r * cos(a), r * sin(a), 0));
}

mobility.SetPositionAllocator (pos);
mobility.Install (nodes);
```

```
  AnimationInterface anim ("star.xml");


  Simulator::Stop (Seconds (10));

  Simulator::Run ();

  Simulator::Destroy ();

  return 0;

}
```

TREE TOPOLOGY



CODE:

```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/internet-module.h"
#include "ns3/netanim-module.h"
#include <vector>

using namespace ns3;

int main(int argc, char *argv[])
{
    Time::SetResolution(Time::NS);

    const uint32_t nRoot = 1;
    const uint32_t nLevel1 = 3;
    const uint32_t nLevel2PerL1 = 5; // 3 * 5 = 15
    const uint32_t totalNodes = nRoot + nLevel1 + (nLevel1 * nLevel2PerL1);

    NodeContainer nodes;
    nodes.Create(totalNodes);

    InternetStackHelper internet;
    internet.Install(nodes);

    PointToPointHelper p2p;
    p2p.SetDeviceAttribute("DataRate", StringValue("10Mbps"));
    p2p.SetChannelAttribute("Delay", StringValue("2ms"));

    Ipv4AddressHelper ipv4;
    ipv4.SetBase("10.4.0.0", "255.255.255.0");

    uint32_t index = 0;

    // Root node
    Ptr<Node> root = nodes.Get(index++);

    // Level 1 nodes
    std::vector<Ptr<Node>> level1;
    for (uint32_t i = 0; i < nLevel1; ++i)
    {
        Ptr<Node> l1 = nodes.Get(index++);
        level1.push_back(l1);
```

```
    NetDeviceContainer d = p2p.Install(NodeContainer(root, l1));
    ipv4.Assign(d);
    ipv4.NewNetwork();
}

// Level 2 nodes
for (uint32_t i = 0; i < nLevel1; ++i)
{
    for (uint32_t j = 0; j < nLevel2PerL1; ++j)
    {
        Ptr<Node> l2 = nodes.Get(index++);
        NetDeviceContainer d = p2p.Install(NodeContainer(level1[i], l2));
        ipv4.Assign(d);
        ipv4.NewNetwork();
    }
}


AnimationInterface anim("outputs/topology4_tree.xml");



// Root position
anim.SetConstantPosition(root, 50.0, 90.0);

// Level 1 positioning
double l1StartX = 30.0;
double l1Gap = 20.0;

for (uint32_t i = 0; i < nLevel1; ++i)
{
    anim.SetConstantPosition(level1[i],
                             l1StartX + i * l1Gap,
                             65.0);
}

// Level 2 positioning (grouped under parents)
uint32_t l2Index = 1 + nLevel1;
double l2Gap = 4.0;

for (uint32_t i = 0; i < nLevel1; ++i)
{
    double parentX = l1StartX + i * l1Gap;
    double childStartX = parentX - (nLevel2PerL1 - 1) * l2Gap / 2;

    for (uint32_t j = 0; j < nLevel2PerL1; ++j)
```

```
        {
            anim.SetConstantPosition(nodes.Get(l2Index++),
                                     childStartX + j * l2Gap,
                                     35.0);
        }
    }

    Simulator::Stop(Seconds(10.0));
    Simulator::Run();
    Simulator::Destroy();

    return 0;
}
```